

CSS Grid Lines Naming

Easy Guide with Examples

Contents

1	Introduction to Grid Lines Naming	2
1.1	What is Grid Lines Naming?	2
1.2	Why Use Named Grid Lines?	2
1.3	Basic Concept	2
2	Syntax for Naming Grid Lines	3
2.1	Basic Naming Syntax	3
2.2	Multiple Names for One Line	3
2.3	Complete Example Structure	3
3	Naming Column Lines	4
3.1	Basic Column Line Naming	4
3.2	Example 1: Sidebar and Main Content	4
3.3	Example 2: Magazine Layout with Multiple Columns	5
4	Naming Row Lines	6
4.1	Basic Row Line Naming	6
4.2	Example 1: Full-Page Layout	6
4.3	Example 2: Image Gallery with Captions	7
5	Combining Named Column and Row Lines	8
5.1	2D Named Grid Lines	8
5.2	Example: Full Dashboard Layout	8
6	Repeat Function with Named Lines	10
6.1	Using repeat() with Names	10
6.2	Generating Multiple Named Lines	10
6.3	Practical Example: 12-Column Bootstrap-like Grid	11
7	Advanced Naming Techniques	12
7.1	Naming with Multiple Names at Same Line	12
7.2	Semantic Naming Convention	12
7.3	Responsive Named Lines	13
8	Named Lines with Auto-Placement	14
8.1	How Auto-Placement Uses Named Lines	14
8.2	Implicit Named Line Indices	14

9	Best Practices for Named Lines	16
9.1	Do's	16
9.2	Don'ts	16
10	Common Mistakes	17
10.1	Mistake 1: Inconsistent Naming	17
10.2	Mistake 2: Forgetting the Brackets	17
10.3	Mistake 3: Using Invalid Characters	17
10.4	Mistake 4: Referencing Non-Existent Names	18
11	Real-World Examples	19
11.1	Example 1: Complete Website Layout	19
11.2	Example 2: E-Commerce Product Page	20
11.3	Example 3: Blog Article Template	20
12	Comparison: Named Lines vs Named Areas	22
12.1	Named Lines	22
12.2	Named Areas	22
12.3	When to Use Which	23
13	Practice Exercises	24
13.1	Exercise 1: Simple Named Grid	24
13.2	Exercise 2: Dashboard with Named Lines	24
13.3	Exercise 3: Responsive Layout	24
13.4	Exercise 4: Nested Named Grids	25
14	Summary	26
14.1	Key Takeaways	26
14.2	Quick Reference	26
14.3	Next Steps	27

Chapter 1

Introduction to Grid Lines Naming

CSS Grid Lines Naming is a powerful feature that allows you to assign custom names to grid lines instead of using numeric indices. This makes your CSS code more readable, maintainable, and self-documenting.

1.1 What is Grid Lines Naming?

By default, grid lines are numbered 1, 2, 3, etc. With naming, you can assign meaningful names like "sidebar-start", "content-end", "footer-line", making your code much clearer and easier to understand.

1.2 Why Use Named Grid Lines?

- **Readability:** Code speaks for itself - "sidebar-end" is clearer than "2"
- **Maintainability:** Easier to update layouts without remembering line numbers
- **Documentation:** Names document the purpose of each grid track
- **Error Reduction:** Less chance of using wrong line numbers
- **Collaboration:** Team members understand layout intent immediately
- **Scalability:** Complex grids become manageable with named lines

1.3 Basic Concept

Instead of:

```
grid-column: 1 / 3;
```

You can write:

```
grid-column: sidebar-start / content-start;
```

Both do the same thing, but the second is immediately understandable.

Chapter 2

Syntax for Naming Grid Lines

2.1 Basic Naming Syntax

```
grid-template-columns:  
  [line-name] 200px [another-name] 1fr [last-name];
```

Names go in square brackets before or after the track size.

2.2 Multiple Names for One Line

A single grid line can have multiple names:

```
grid-template-columns:  
  [sidebar-start nav-start] 200px  
  [sidebar-end nav-end content-start] 1fr  
  [content-end];
```

This is useful when different components reference the same line differently.

2.3 Complete Example Structure

```
.container {  
  display: grid;  
  grid-template-columns:  
    [nav-start] 200px  
    [nav-end content-start] 1fr  
    [content-end];  
  
  grid-template-rows:  
    [header-start] 60px  
    [header-end body-start] 1fr  
    [body-end footer-start] 60px  
    [footer-end];  
}
```

Chapter 3

Naming Column Lines

3.1 Basic Column Line Naming

```
.container {  
  display: grid;  
  grid-template-columns:  
    [col1-start] 100px  
    [col1-end col2-start] 100px  
    [col2-end col3-start] 100px  
    [col3-end];  
}  
  
.item {  
  grid-column: col1-start / col2-end;  
}
```

3.2 Example 1: Sidebar and Main Content

```
HTML:  
<div class="container">  
  <aside>Sidebar</aside>  
  <main>Main Content</main>  
</div>  
  
CSS:  
.container {  
  display: grid;  
  grid-template-columns:  
    [sidebar-start] 250px  
    [sidebar-end main-start] 1fr  
    [main-end];  
}  
  
aside {  
  grid-column: sidebar-start / sidebar-end;  
}
```

```
main {  
  grid-column: main-start / main-end;  
}
```

3.3 Example 2: Magazine Layout with Multiple Columns

```
.magazine {  
  display: grid;  
  grid-template-columns:  
    [full-start] 1fr  
    [col1-start] 1fr  
    [col1-end col2-start] 1fr  
    [col2-end col3-start] 1fr  
    [col3-end full-end] 1fr;  
}  
  
.hero {  
  grid-column: full-start / full-end; /* Full width */  
}  
  
.feature {  
  grid-column: col1-start / col3-end; /* 3 columns */  
}  
  
.article {  
  grid-column: col2-start / col2-end; /* 1 column */  
}
```

Chapter 4

Naming Row Lines

4.1 Basic Row Line Naming

```
.container {  
  display: grid;  
  grid-template-rows:  
    [header-start] 60px  
    [header-end main-start] 1fr  
    [main-end footer-start] 60px  
    [footer-end];  
}  
  
header {  
  grid-row: header-start / header-end;  
}  
  
main {  
  grid-row: main-start / main-end;  
}  
  
footer {  
  grid-row: footer-start / footer-end;  
}
```

4.2 Example 1: Full-Page Layout

```
HTML:  
<div class="page">  
  <header>Header</header>  
  <nav>Navigation</nav>  
  <main>Content</main>  
  <footer>Footer</footer>  
</div>  
  
CSS:  
.page {  
  display: grid;
```



```
grid-template-rows:
  [header-start] 60px
  [header-end nav-start] 50px
  [nav-end main-start] 1fr
  [main-end footer-start] 60px
  [footer-end];
height: 100vh;
}

header {
  grid-row: header-start / header-end;
}

nav {
  grid-row: nav-start / nav-end;
}

main {
  grid-row: main-start / main-end;
}

footer {
  grid-row: footer-start / footer-end;
}
```

4.3 Example 2: Image Gallery with Captions

```
.gallery {
  display: grid;
  grid-template-rows:
    [image-start] 300px
    [image-end caption-start] 80px
    [caption-end];
  grid-template-columns: repeat(3, 1fr);
}

img {
  grid-row: image-start / image-end;
}

figcaption {
  grid-row: caption-start / caption-end;
}
```

Chapter 5

Combining Named Column and Row Lines

5.1 2D Named Grid Lines

Combine column and row names for precise positioning:

```
.container {
  display: grid;
  grid-template-columns:
    [nav-start] 200px
    [nav-end main-start] 1fr
    [main-end];

  grid-template-rows:
    [header-start] 60px
    [header-end body-start] 1fr
    [body-end];
}

.header {
  grid-column: nav-start / main-end;
  grid-row: header-start / header-end;
}

.sidebar {
  grid-column: nav-start / nav-end;
  grid-row: body-start / body-end;
}

.main {
  grid-column: main-start / main-end;
  grid-row: body-start / body-end;
}
```

5.2 Example: Full Dashboard Layout

HTML:

```
<div class="dashboard">
  <header>Dashboard</header>
  <aside>Menu</aside>
  <main>Content</main>
  <footer>Status</footer>
</div>
```

CSS:

```
.dashboard {
  display: grid;
  grid-template-columns:
    [sidebar-start] 200px
    [sidebar-end content-start] 1fr
    [content-end];

  grid-template-rows:
    [header-start] 50px
    [header-end main-start] 1fr
    [main-end footer-start] 40px
    [footer-end];
}

header {
  grid-column: sidebar-start / content-end;
  grid-row: header-start / header-end;
  background: #333;
}

aside {
  grid-column: sidebar-start / sidebar-end;
  grid-row: main-start / main-end;
  background: #f5f5f5;
}

main {
  grid-column: content-start / content-end;
  grid-row: main-start / main-end;
}

footer {
  grid-column: sidebar-start / content-end;
  grid-row: footer-start / footer-end;
  background: #333;
}
```

Chapter 6

Repeat Function with Named Lines

6.1 Using repeat() with Names

The `repeat()` function works with named lines:

```
.container {
  display: grid;
  grid-template-columns:
    [full-start] 1fr
    repeat(3, [col-start] 100px [col-end])
    [full-end];
}

/* This creates:
   [full-start] 1fr [col-start] 100px [col-end]
   [col-start] 100px [col-end] [col-start] 100px [col-end]
   [full-end]
*/
```

6.2 Generating Multiple Named Lines

```
.grid {
  display: grid;
  grid-template-columns:
    repeat(12, [col-start] 1fr [col-end]);
}

.item1 {
  grid-column: col-start 1 / col-end 3; /* Cols 1-3 */
}

.item2 {
  grid-column: col-start 4 / col-end 9; /* Cols 4-9 */
}

.item3 {
  grid-column: col-start 10 / col-end 12; /* Cols 10-12 */
}
```

6.3 Practical Example: 12-Column Bootstrap-like Grid

```
.bootstrap-grid {  
  display: grid;  
  grid-template-columns:  
    repeat(12, [col-start] 1fr [col-end]);  
  gap: 20px;  
}  
  
.col-1 { grid-column: col-start 1 / col-end 1; }  
.col-2 { grid-column: col-start 1 / col-end 2; }  
.col-3 { grid-column: col-start 1 / col-end 3; }  
.col-6 { grid-column: col-start 1 / col-end 6; }  
.col-12 { grid-column: col-start 1 / col-end 12; }
```

Chapter 7

Advanced Naming Techniques

7.1 Naming with Multiple Names at Same Line

```
.container {
  display: grid;
  grid-template-columns:
    [nav-start menu-start] 200px
    [nav-end menu-end main-start content-start] 1fr
    [main-end content-end];
}

/* Use any name to reference the line */
.menu {
  grid-column: menu-start / menu-end;
}

.main {
  grid-column: main-start / main-end;
}

.content {
  grid-column: content-start / content-end;
}
```

7.2 Semantic Naming Convention

```
.layout {
  display: grid;
  grid-template-columns:
    [page-start container-start] 20px
    [content-start] 1fr
    [content-end] 20px
    [page-end];

  grid-template-rows:
    [page-start] auto
    [hero-start] 400px
```

```
    [hero-end body-start] 1fr
    [body-end footer-start] 200px
    [footer-end page-end];
}

/* Self-documenting CSS */
.page-wrapper {
    grid-column: page-start / page-end;
    grid-row: page-start / page-end;
}

.hero {
    grid-column: container-start / page-end;
    grid-row: hero-start / hero-end;
}

.content {
    grid-column: content-start / content-end;
    grid-row: body-start / body-end;
}
```

7.3 Responsive Named Lines

```
/* Mobile */
.container {
    display: grid;
    grid-template-columns:
        [full-start] 1fr [full-end];
}

/* Tablet */
@media (min-width: 768px) {
    .container {
        grid-template-columns:
            [nav-start] 200px
            [nav-end main-start] 1fr
            [main-end];
    }
}

/* Desktop */
@media (min-width: 1024px) {
    .container {
        grid-template-columns:
            [full-start] 20px
            [sidebar-start] 250px
            [sidebar-end main-start] 1fr
            [main-end] 20px
            [full-end];
    }
}
```

Chapter 8

Named Lines with Auto-Placement

8.1 How Auto-Placement Uses Named Lines

```
.container {
  display: grid;
  grid-template-columns:
    [col-start] 1fr
    [col-start] 1fr
    [col-start] 1fr;
  grid-auto-rows: 100px;
}

/* Auto-placement uses 'col-start' lines */
.item:nth-child(1) {
  grid-column: col-start 1; /* First col-start */
}

.item:nth-child(2) {
  grid-column: col-start 2; /* Second col-start */
}

.item:nth-child(3) {
  grid-column: col-start 3; /* Third col-start */
}
```

8.2 Implicit Named Line Indices

```
.container {
  display: grid;
  grid-template-columns:
    repeat(4, [track-start] 1fr [track-end]);
}

.item {
  /* Reference specific occurrence of name */
  grid-column: track-start 2 / track-end 3;
  /* Spans from second track-start to third track-end */
}
```



```
}
```

Chapter 9

Best Practices for Named Lines

9.1 Do's

- **Use semantic names:** "sidebar-start" not "line1"
- **Follow conventions:** Use consistent naming patterns
- **Include context:** Names should indicate purpose
- **Use descriptive terms:** "nav-end" is better than "end"
- **Document your grid:** Add comments explaining layout
- **Keep names reasonable:** Not too long or cryptic
- **Use hyphens:** Follow CSS naming conventions
- **Group related names:** "header-start/end", "main-start/end"

9.2 Don'ts

- **Don't use vague names:** "section1", "area2" lack context
- **Don't mix naming schemes:** Stay consistent throughout
- **Don't use numbers:** Names, not numbers, are the point
- **Don't overcomplicate:** Keep naming simple and clear
- **Don't ignore readability:** Code clarity is crucial
- **Don't repeat unnecessarily:** Reuse names when appropriate

Chapter 10

Common Mistakes

10.1 Mistake 1: Inconsistent Naming

```
/* WRONG - Inconsistent naming patterns */
.container {
  grid-template-columns:
    [sidebar-start] 200px
    [right] 1fr; /* Wrong pattern - "right" instead of "sidebar-end" */
}

/* CORRECT */
.container {
  grid-template-columns:
    [sidebar-start] 200px
    [sidebar-end] 1fr;
}
```

10.2 Mistake 2: Forgetting the Brackets

```
/* WRONG - No brackets around name */
.container {
  grid-template-columns: sidebar 200px main 1fr;
}

/* CORRECT */
.container {
  grid-template-columns:
    [sidebar] 200px
    [main] 1fr;
}
```

10.3 Mistake 3: Using Invalid Characters

```
/* WRONG - Spaces and special characters */
.container {
```

```
grid-template-columns:
  [side bar start] 200px; /* Spaces not allowed */
}

/* CORRECT */
.container {
  grid-template-columns:
    [sidebar-start] 200px;
}
```

10.4 Mistake 4: Referencing Non-Existent Names

```
/* WRONG - Using name that doesn't exist */
.container {
  grid-template-columns:
    [sidebar-start] 200px
    [main-start] 1fr;
}

.item {
  grid-column: content-start / content-end; /* These names don't exist */
}

/* CORRECT */
.item {
  grid-column: sidebar-start / main-start;
}
```

Chapter 11

Real-World Examples

11.1 Example 1: Complete Website Layout

```
HTML:
<div class="website">
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</div>

CSS:
.website {
  display: grid;

  grid-template-columns:
    [full-start] 20px
    [nav-start] 200px
    [nav-end content-start] 1fr
    [content-end] 20px
    [full-end];

  grid-template-rows:
    [header-start] 60px
    [header-end nav-body-start] 1fr
    [nav-body-end footer-start] 100px
    [footer-end];

  min-height: 100vh;
}

header {
  grid-column: full-start / full-end;
  grid-row: header-start / header-end;
  background: #333;
}

nav {
```

```
    grid-column: nav-start / nav-end;
    grid-row: nav-body-start / nav-body-end;
    background: #f5f5f5;
}

main {
    grid-column: content-start / content-end;
    grid-row: nav-body-start / nav-body-end;
}

footer {
    grid-column: full-start / full-end;
    grid-row: footer-start / footer-end;
    background: #333;
}
```

11.2 Example 2: E-Commerce Product Page

```
CSS:
.product-page {
    display: grid;

    grid-template-columns:
        [page-start gutter-start] 20px
        [gutter-end images-start] 400px
        [images-end info-start] 1fr
        [info-end gutter-start] 20px
        [gutter-end page-end];

    grid-template-rows:
        [gallery-start] auto
        [gallery-end details-start] auto
        [details-end];
}

.product-images {
    grid-column: images-start / images-end;
    grid-row: gallery-start / details-end;
}

.product-info {
    grid-column: info-start / info-end;
    grid-row: gallery-start / details-end;
}
```

11.3 Example 3: Blog Article Template

```
CSS:
.article-layout {
    display: grid;
```

```
grid-template-columns:
  [full-start container-start] 1fr
  [text-start] 700px
  [text-end container-end full-end] 1fr;

grid-template-rows:
  [hero-start] 400px
  [hero-end content-start] auto
  [content-end];
}

.hero {
  grid-column: full-start / full-end;
  grid-row: hero-start / hero-end;
}

article {
  grid-column: text-start / text-end;
  grid-row: content-start / content-end;
}
```

Chapter 12

Comparison: Named Lines vs Named Areas

12.1 Named Lines

```
.container {
  grid-template-columns:
    [nav-start] 200px
    [nav-end content-start] 1fr
    [content-end];
}

.nav {
  grid-column: nav-start / nav-end;
}
```

Advantages:

- More granular control
- Can reference individual lines
- Flexible positioning

12.2 Named Areas

```
.container {
  grid-template-areas:
    "nav content"
    "nav content";
}

.nav {
  grid-area: nav;
}
```

Advantages:

- Visual representation of layout

- Easier to see overall structure
- Quick layout changes

12.3 When to Use Which

- Use **named lines** for:
 - Complex, asymmetrical layouts
 - Fine-grained positioning control
 - Multiple items sharing boundaries
- Use **named areas** for:
 - Simple, symmetric layouts
 - Visual layout clarity
 - Responsive layout variations

Chapter 13

Practice Exercises

13.1 Exercise 1: Simple Named Grid

Create a 3-column layout with named lines:

- Column 1: sidebar (200px)
- Column 2: main (1fr)
- Column 3: extra (150px)

Name the lines and position 3 items using the names.

13.2 Exercise 2: Dashboard with Named Lines

Build a dashboard layout with named lines:

- Header row
- Sidebar column
- Main content area
- Footer row

Use semantic naming throughout.

13.3 Exercise 3: Responsive Layout

Create a layout that:

- Uses different named lines at mobile (single column)
- Uses different named lines at tablet (two columns)
- Uses different named lines at desktop (three columns)

13.4 Exercise 4: Nested Named Grids

Create two grids with named lines:

- Outer container with named lines
- Inner container with its own named lines
- Combine them in a layout

Chapter 14

Summary

14.1 Key Takeaways

- **Named lines** make grid positioning readable and maintainable
- **Syntax:** Use square brackets around names in grid templates
- **Column lines:** Name columns with grid-template-columns
- **Row lines:** Name rows with grid-template-rows
- **Multiple names:** A single line can have multiple names
- **repeat() support:** Names work with the repeat() function
- **Semantic naming:** Use meaningful, self-documenting names
- **Responsiveness:** Change named lines at different breakpoints
- **Consistency:** Maintain naming patterns throughout projects

14.2 Quick Reference

```
/* Naming columns */
grid-template-columns:
  [start] 200px
  [end main-start] 1fr
  [main-end];

/* Naming rows */
grid-template-rows:
  [header] 60px
  [body] 1fr
  [footer] 60px;

/* Using named lines */
grid-column: start / end;
grid-row: header / body;
```

```
/* Multiple names at same line */  
[nav-start sidebar-start] 200px  
  
/* With repeat() */  
repeat(3, [col-start] 1fr [col-end])  
  
/* Referencing repeated names */  
grid-column: col-start 1 / col-start 3;
```

14.3 Next Steps

Explore related Grid topics:

- CSS Grid Area Naming - Template-based named areas
- CSS Grid Items Positioning - Precise placement techniques
- CSS Grid Items Spanning - Making items span multiple cells
- Responsive Grid Layouts - Multi-breakpoint strategies